

# Unidad 3

**Diseño de algoritmos con  
diagramas de flujo y  
pseudocódigo... y algo de Java**

## Tabla de contenido

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1. Introducción.....</b>                                  | <b>3</b>  |
| <b>2. Estructura de un algoritmo.....</b>                    | <b>3</b>  |
| <b>2.1. Instrucciones para la declaración de datos. ....</b> | <b>3</b>  |
| <b>2.2. Instrucciones simples y compuestas.....</b>          | <b>4</b>  |
| Asignación .....                                             | 4         |
| Instrucciones simples .....                                  | 4         |
| Entrada .....                                                | 5         |
| Salida .....                                                 | 5         |
| Instrucciones compuestas .....                               | 6         |
| <b>2.3. Estructuras de control .....</b>                     | <b>12</b> |
| 2.3.1. La estructura secuencia. ....                         | 12        |
| 2.3.2. Estructuras de decisión .....                         | 13        |
| <b>2.4. Estructuras iterativas (bucles).....</b>             | <b>26</b> |
| 2.4.1. Estructura WHILE .....                                | 26        |
| 2.4.2. Estructura DO-WHILE. ....                             | 27        |
| 2.4.3. Estructura FOR.....                                   | 28        |
| 2.4.4. Finalización de un bucle .....                        | 29        |
| <b>2.5. Variables de control .....</b>                       | <b>30</b> |
| 2.5.1. Contadores. ....                                      | 30        |
| 2.5.2. Acumuladores .....                                    | 33        |
| 2.5.3. Interruptores. ....                                   | 36        |

# 1. Introducción.

En esta unidad aprenderemos a diseñar algoritmos utilizando dos técnicas diferentes: diagramas de flujo y pseudocódigo, y utilizaremos algo de Java para programar algunos de los algoritmos y verlos funcionando en un ordenador real.

Aunque no usaremos diagramas de flujo en los próximos temas, son muy útiles para aprender algunas de las estructuras de control y para seguir el flujo de ejecución de algunos algoritmos. Así que, aprenderemos a usarlos junto con el pseudocódigo, que usaremos con más frecuencia cuando diseñemos algoritmos en el futuro.

## 2. Estructura de un algoritmo

En el diseño y desarrollo de algoritmos, podemos distinguir diferentes tipos de instrucciones.

Nos ocuparemos de ellos en detalle y veremos su aplicación resolviendo ciertos problemas.

- Instrucciones para la declaración de datos.
- Instrucciones simples y compuestas.
- Estructuras de control.

### 2.1. Instrucciones para la declaración de datos.

Son aquellas instrucciones que indican al ordenador el espacio que debe reservarse en la memoria principal para almacenar la información que manejará el programa:

`<tipo> <identificador>`

Ejemplo: `int age`

Esta declaración, aunque conveniente tanto en diagramas de flujo como en pseudocódigo, en general al diseñar diagramas de flujo, no la realizamos.

## 2.2. Instrucciones simples y compuestas

### Instrucciones simples

Las instrucciones simples son: asignación, entrada y salida.



#### Asignación

Instrucciones de asignación: son instrucciones que almacenan datos de una expresión y los almacenan en una variable previamente definida y declarada en el algoritmo. Usamos el símbolo igual =

|                                                                                                              |                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><u>Pseudocode</u></b><br><br>LONG = 3.51<br>counter = 10<br>name = "Antonio"<br>sentence = "Good morning" | <b><u>Flowchart</u></b><br><br><pre>graph TD; Start(( )) --&gt; Process[\"LONG = 3.51&lt;br/&gt;counter = 10&lt;br/&gt;name = 'Antonio'&lt;br/&gt;sentence = 'Good morning'\"]; Process --&gt; End(( ))</pre> |
| <b><u>Pseudocode</u></b><br><br>counter = 10 + 1<br>counter = counter + 1                                    | <b><u>Flowchart</u></b><br><br><pre>graph TD; Start(( )) --&gt; Process1[\"counter = 10 + 1\"]; Process1 --&gt; Process2[\"counter = counter + 1\"]; Process2 --&gt; End(( ))</pre>                           |



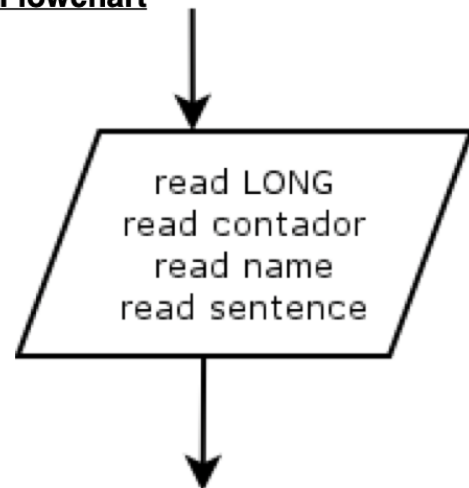
## Entrada

Las Instrucciones de Entrada son aquellas que recogen los datos de un dispositivo o dispositivo de entrada y los almacenan en la memoria en una variable previamente definida en el algoritmo.

### Pseudocode

```
read LONG //Expects a Real Numeric
read counter //Expects an Integer Numeric
read name //Expects an alphanumeric
read sentence //Expects an alphanumeric
```

### Flowchart



## Salida

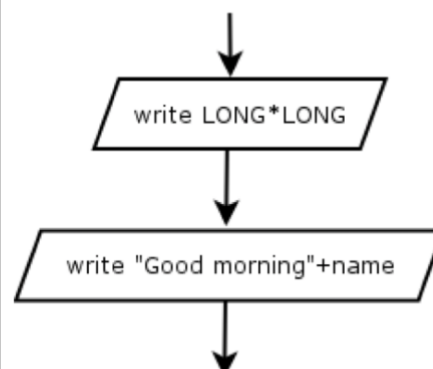
Instrucciones de salida: se utilizan para enviar los datos contenidos en las variables o los resultados de cualquier expresión a un dispositivo de salida.

### Pseudocode

```
write LONG * LONG
//Displays the result of the expresion
//on the screen

write "Good morning " + name
//Displays "Good morning" and then
//the content of the variable name
```

### Flowchart



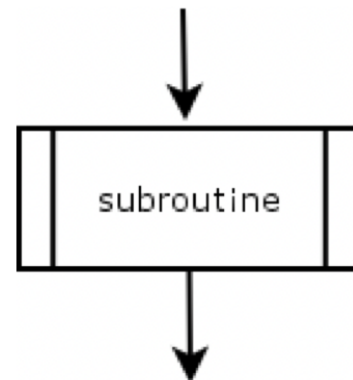
### Instrucciones compuestas

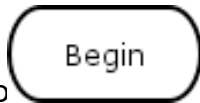
Las instrucciones compuestas son aquellas que forman un bloque de acciones agrupadas en subrutinas, subprogramas, funciones o módulos.

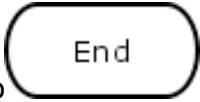
#### Pseudocode

```
subroutine  
//Just the name of the subroutine
```

#### Flowchart



Los diagramas de flujo siempre comienzan con el símbolo  en el lugar más alto del diagrama de flujo

Y termina con el símbolo  en la parte inferior del diagrama de flujo

Ejemplos lineales:

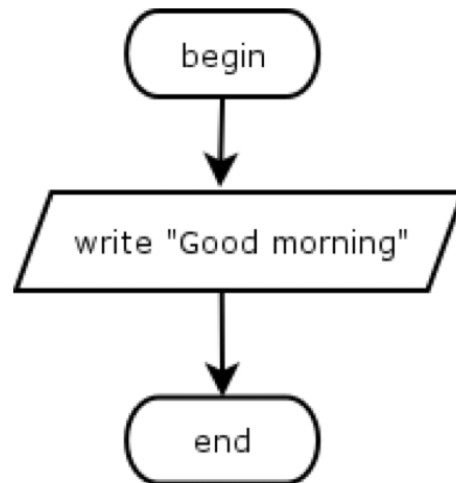
Ejemplo 1: Diseña un algoritmo que muestre “Good morning”:

e.g. 1: Design an algorithm that displays “Good morning”:

**Pseudocode**

write “Good morning”

**Flowchart**



En Java:

```
class Example1 {  
    public static void main(String[] argv) {  
        System.out.println("Good morning");  
    }  
}
```

Crear un proyecto Java Example1 en IntelliJ IDEA y subirlo a Github (completar la actividad en la clase virtual)

**NOTA: No te preocupes en este momento por lo que significan las instrucciones de Java. Los aprenderás más adelante. Ahora solo tienes que aprender la parte del algoritmo del programa Java.**

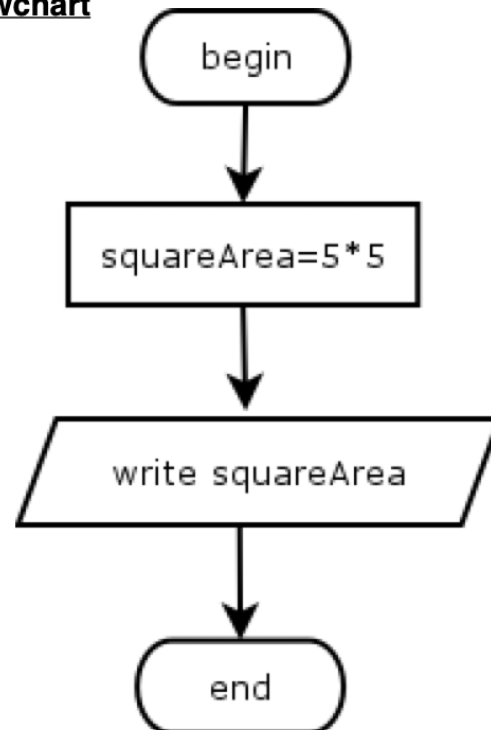
Ejemplo 2: Diseña un algoritmo que calcule y muestre el área de un cuadrado cuyo lado mide 5.

e.g. 2: Design an algorithm that calculates and displays the area of a square whose side measures 5.

**Pseudocode**

```
int squareArea  
squareArea = 5 * 5  
write squareArea
```

**Flowchart**



```
class Example2 {  
    public static void main(String[] argv) {  
        int squareArea = 5 * 5;  
        System.out.println(squareArea);  
    }  
}
```

Crea un proyecto Java Example2 en IntelliJ IDEA y súbelos a Github (completa la actividad en el aula virtual)



Ejemplo 3: Diseña un algoritmo que calcule y muestre el área de un cuadrado cuyo lado se introduce mediante el teclado

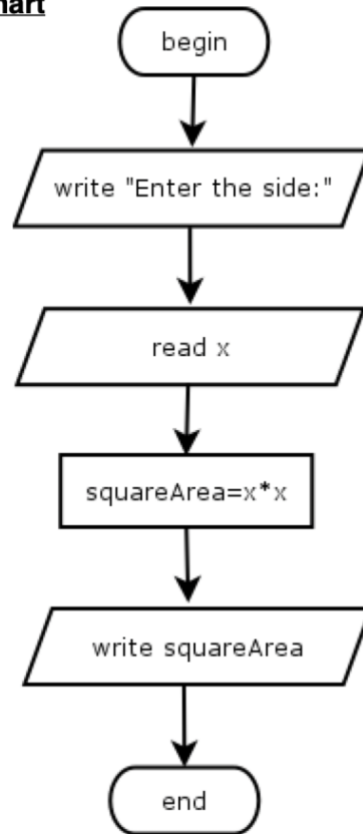
e.g. 3: Design an algorithm that calculates and displays the area of a square whose side is entered by keyboard

#### **Pseudocode**

```
int squareArea
int x

write "Enter the side:"
read x
squareArea = x * x
write squareArea
```

#### **Flowchart**



```
import java.util.Scanner;
class Example3 {
    public static void main(String[] argv) {
        float x;
        float squareArea;
        System.out.println("Enter the side:");

        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        x = inputValue.nextFloat();

        squareArea = x * x;
        System.out.println(squareArea);
    }
}
```

Ejercicios para que hagas:

4.- Diseña un algoritmo que lea dos números y muestre el resultado de sumarlos, restarlos, multiplicarlos y dividirlos.

5.- Diseña un algoritmo que lea el radio de un círculo y muestre su longitud y área.

Ejemplo 6: Diseña un algoritmo que lea el precio de venta al público de un artículo y su precio real y calcule el descuento como un %.

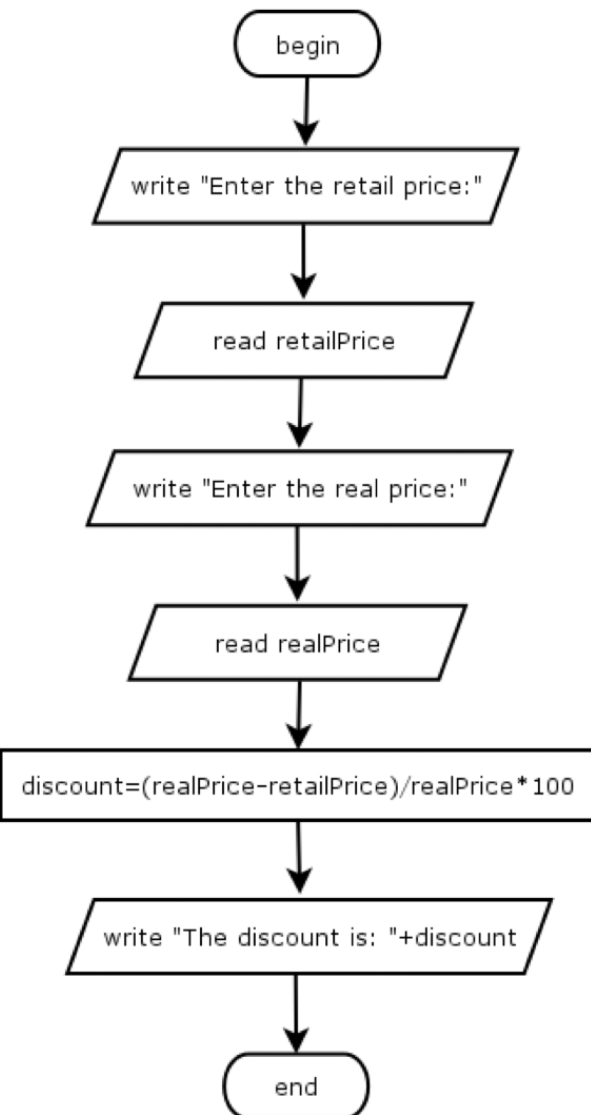
e.g. 6: Design an algorithm that reads the retail price of an article and its real price and calculates the discount as a %.

### **Pseudocode**

```
float retailPrice
float realPrice
float discount

write "Enter the retail price:"
read retailPrice
write "Enter the real price:"
read realPrice
discount = (retailPrice - realPrice) /
    retailPrice * 100
write "The discount is: " + discount
```

### **Flowchart**



```
import java.util.Scanner;
class Example6 {
    public static void main(String[] argv) {
        float retailPrice;
        float realPrice;
        float discount;
        System.out.println("Enter the retail price:");

        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        retailPrice = inputValue.nextFloat();

        System.out.println("Enter the real price:");
        realPrice = inputValue.nextFloat();

        discount = (retailPrice - realPrice) / retailPrice * 100;
        System.out.println("The discount is: " + discount + "%");
    }
}
```

Ejercicio para ti:

7.- Diseña un algoritmo que lea el valor de una distancia en millas náuticas y muestre la distancia en metros. 1 milla náutica = 1,852 metros

## 2.3. Estructuras de control

Las instrucciones de control se utilizan para controlar la secuencia de flujo de un programa. Se pueden clasificar en:

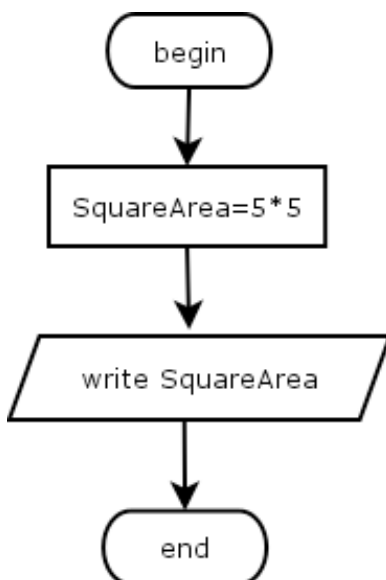
- Estructura de **secuencia**
- Estructuras de **decisión** (simples, dobles y múltiples)
- Estructuras de **repetición** (while, do-while y for).

### 2.3.1. La estructura secuencia.



#### Estructura **secuencia**

Ya lo has estado usando. Consiste en una serie de instrucciones que se realizan en orden una tras otra.



En este caso, la instrucción `squareArea=5*5` se realiza en primer lugar y luego la siguiente instrucción: `write squareArea`

```
class Example2 {  
    public static void main(String[] argv) {  
        int squareArea = 5 * 5;  
        System.out.println(squareArea);  
    }  
}
```

## 2.3.2. Estructuras de decisión



### Estructuras de **decisión**

Las estructuras de decisión controlan la ejecución o no de un conjunto de instrucciones en función del cumplimiento de una determinada condición.

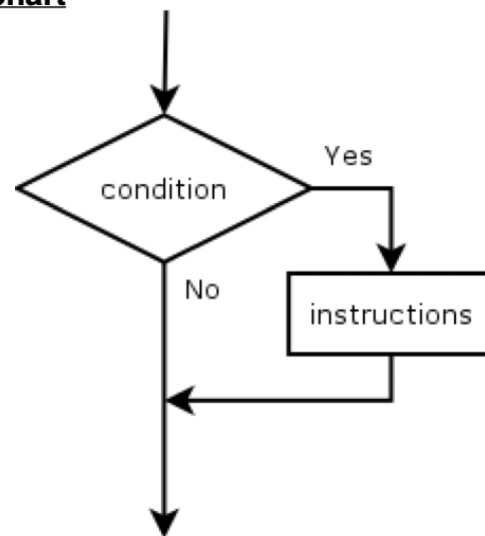
#### **Decisión simple**

En esta estructura se evalúa una condición como VERDADERA o FALSA y se ejecuta un conjunto de instrucciones en función de este valor (verdadero o falso)

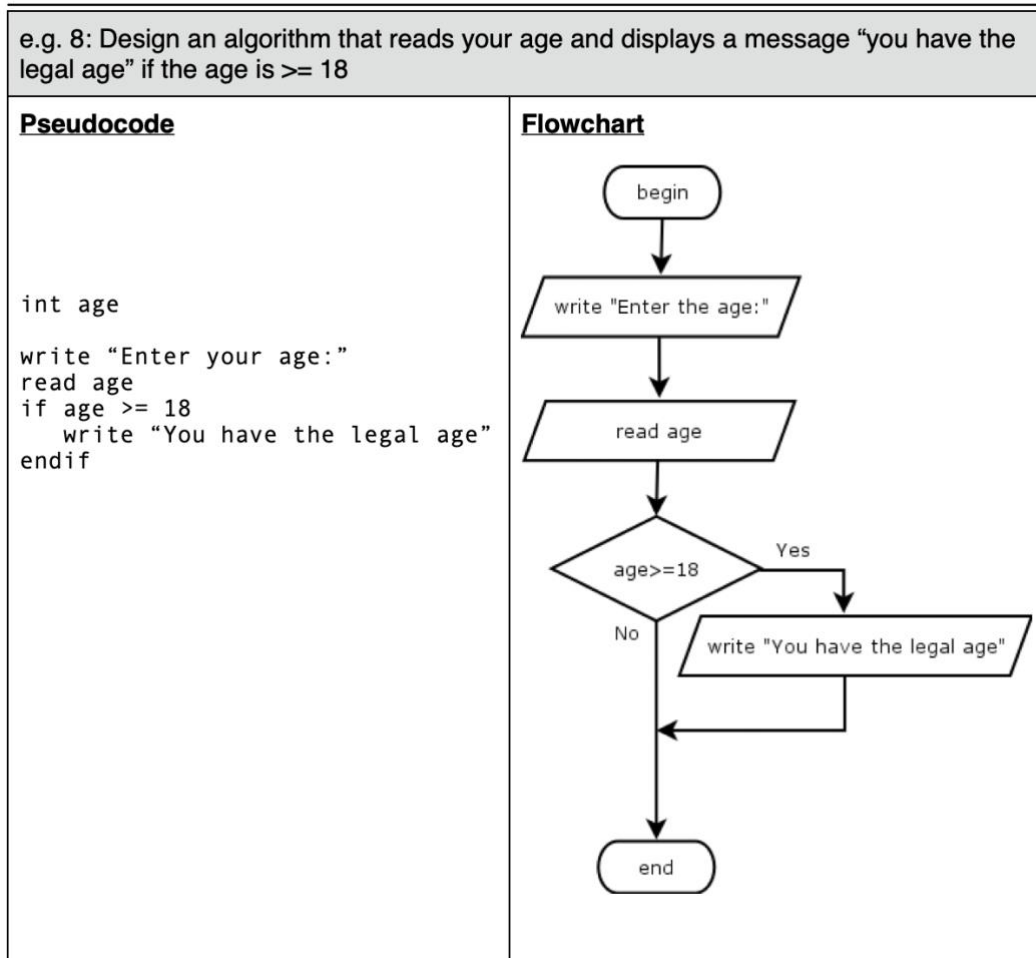
##### Pseudocode

```
if condition
  instruction 1
  instruction 2
  ...
  instruction n
endif
```

##### Flowchart



Ejemplo 8: Diseña un algoritmo que lea la edad y muestre un mensaje "tiene la edad legal" si la edad es  $\geq 18$



```

import java.util.Scanner;
class Example8 {
    public static void main(String[] argv) {
        int age;
        System.out.println("Enter your age:");

        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        age = inputValue.nextInt();

        if (age >= 18) {
            System.out.println("You have the legal age");
        }

    }
}
    
```

### Decisión doble

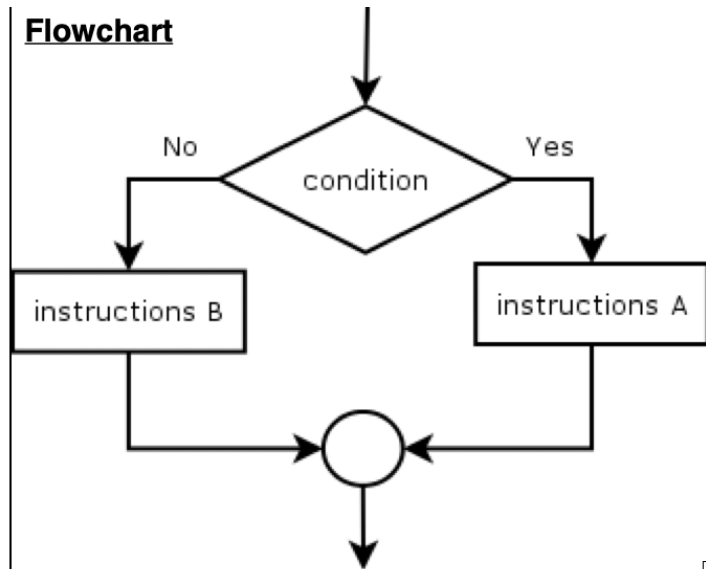
En esta estructura, una condición se evalúa como VERDADERA o FALSO y se ejecuta un conjunto de instrucciones si el resultado es verdadero y se ejecuta otro conjunto si el resultado es falso.

#### Pseudocode

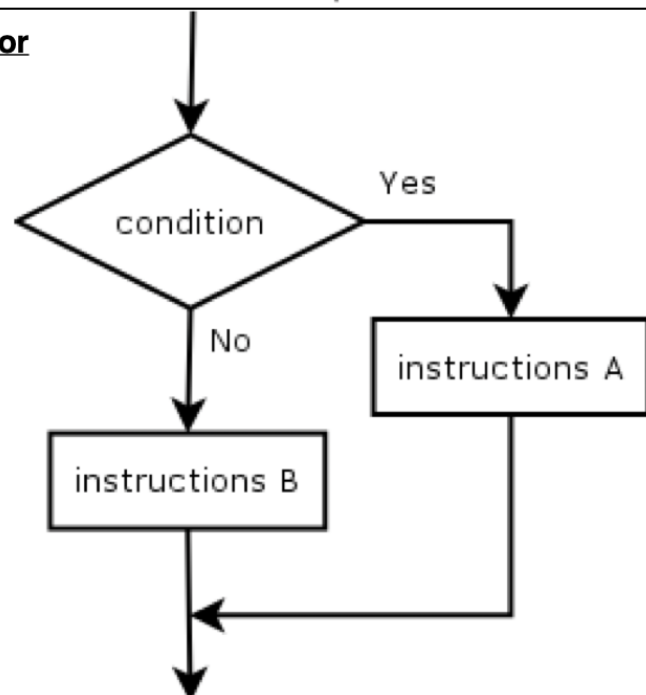
```

if condition
  instruction 1.1
  instruction 1.2
  ...
  instruction 1.n
else
  instruction 2.1
  instruction 2.2
  ...
  instruction 2.m
endif
    
```

#### Flowchart



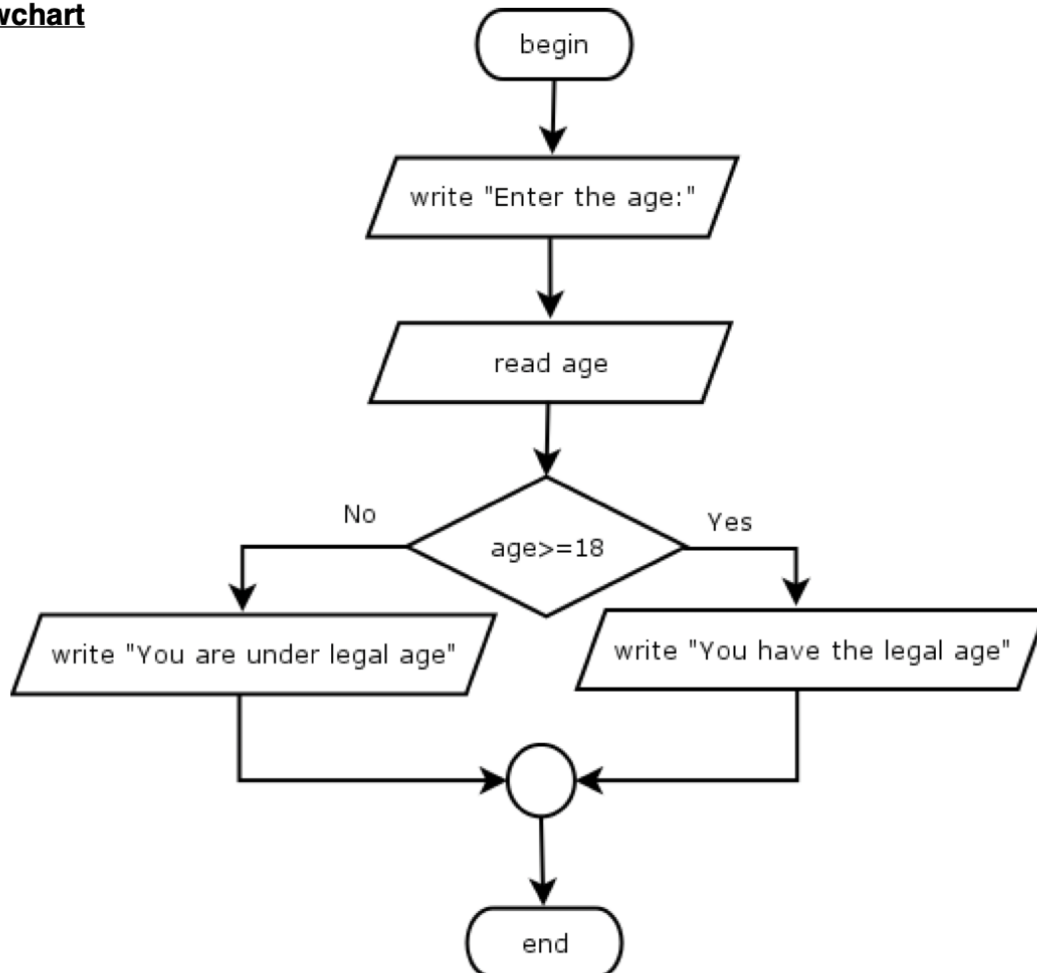
#### or



Ejemplo 9: Diseña un algoritmo que lea la edad y muestre un mensaje "tiene la edad legal" si la edad es  $\geq 18$  y "es menor de edad" en caso contrario.

e.g. 9: Design an algorithm that reads your age and displays a message "you have the legal age" if the age is  $\geq 18$  and "you are under legal age" otherwise.

### **Flowchart**



### **Pseudocode**

```

int age
write "Enter your age:"
read age
if age >= 18
    write "You have the legal age"
else
    write "You are under legal age"
endif
    
```



```
import java.util.Scanner;
class Example9 {
    public static void main(String[] argv) {
        int age;
        System.out.println("Enter your age:");

        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        age = inputValue.nextInt();

        if (age >= 18) {
            System.out.println("You have the legal age:");
        } else {
            System.out.println("You are under legal age:");
        }
    }
}
```

Ejercicios para que hagas:

10.- Diseña un algoritmo que lea un valor y muestre si es positivo o negativo (0 es positivo)

11.- Diseña un algoritmo que lea dos valores y los muestre en orden ascendente.

12.- Diseña un algoritmo que lea dos valores y muestre el mayor de ellos.

13.- Diseña un algoritmo que lea tres valores y muestre el mayor de ellos.

14.- Diseña un algoritmo que lea tres valores y los muestre en orden ascendente.

15.- Diseña un algoritmo que lea un valor numérico entero correspondiente a las notas de un examen y muestre su valor alfanumérico en español:

- de 0 a <3 Muy Deficiente.
- de 3 a <5 Insuficiente.
- de 5 a <6 Suficiente.
- de 6 a <7 Bien
- de 7 a <9 Notable
- de 9 a 10 Sobresaliente

### Decisión alternativa múltiple

En la estructura de decisión alternativa múltiple, se valida una expresión y se ejecuta un conjunto de instrucciones en función del valor resultante de esa expresión.

Si el resultado de la expresión es igual al valor 1 --> Se ejecuta un conjunto de instrucciones

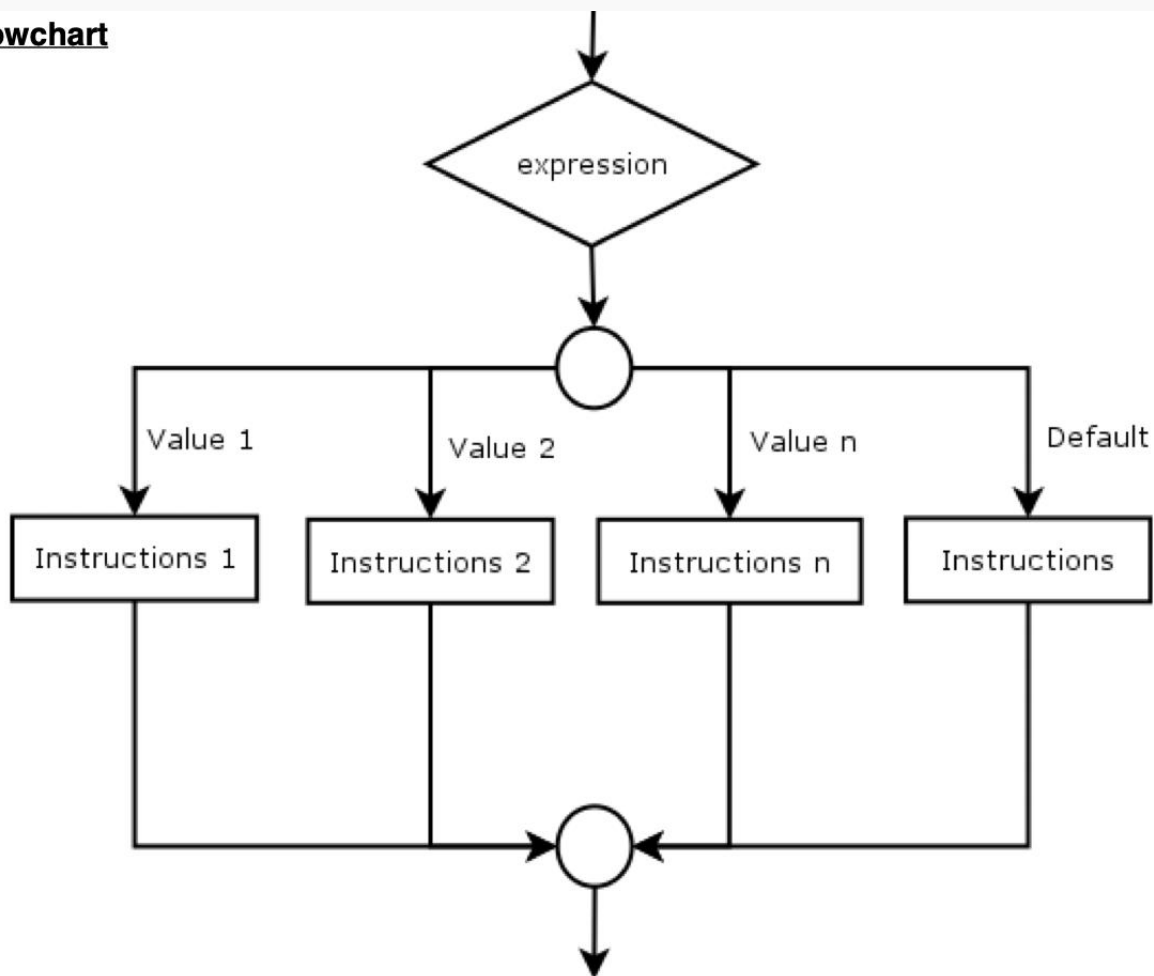
Si el resultado de la expresión es igual al valor 2 --> se ejecuta otro conjunto de instrucciones

...

Si el resultado de la expresión es igual al valor n --> se ejecuta otro conjunto de instrucciones

Y si el resultado no coincide con ninguno de los valores, se ejecuta el conjunto de instrucciones asignadas a "default". Esta cláusula predeterminada es opcional.

### Flowchart



### Pseudocode

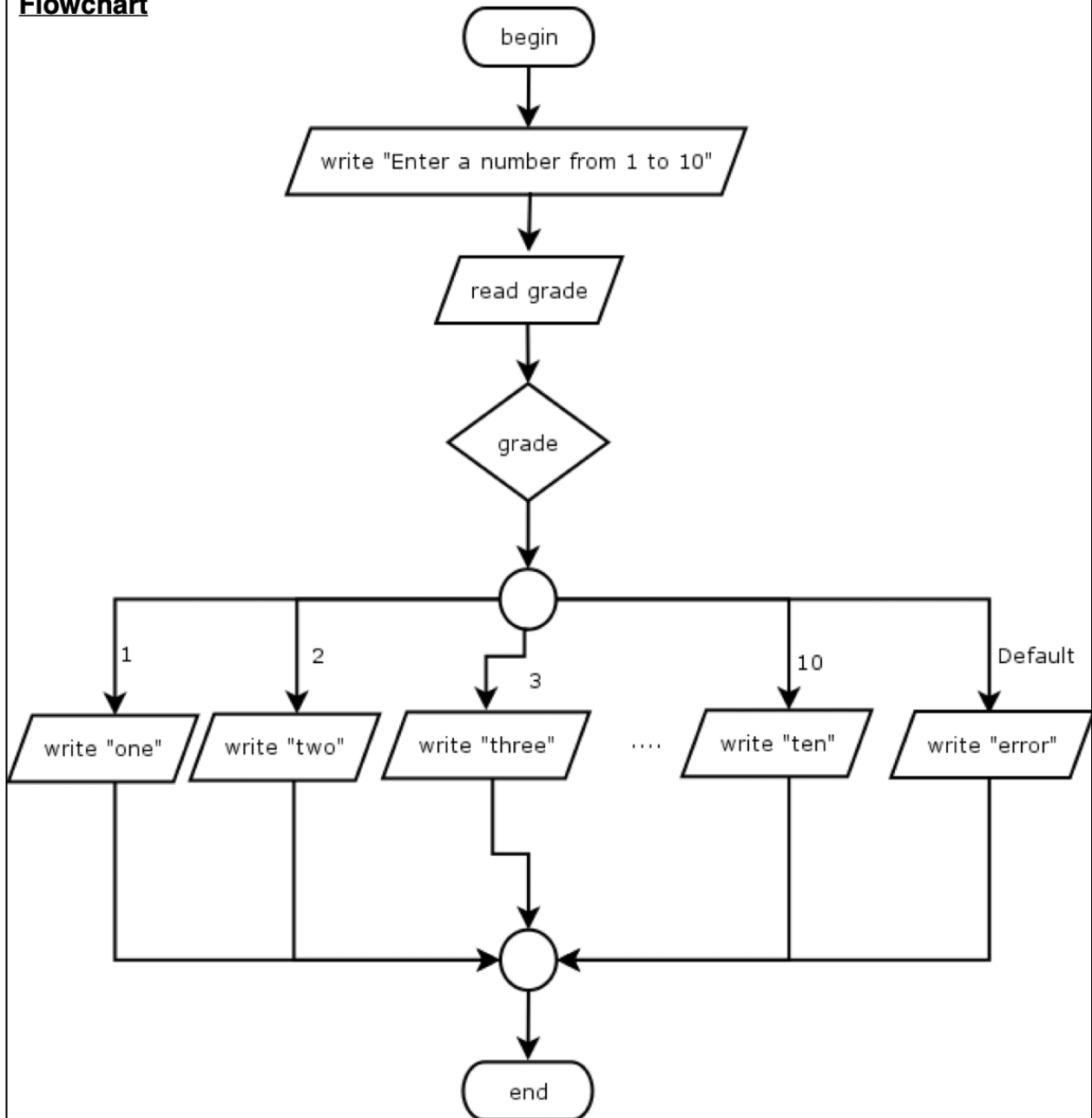
```

case expression
  value 1:
    set of instructions 1
  value 2:
    set of instructions 2
  ...
  value n:
    set of instructions n
  default:
    instruction n
endcase
    
```

Ejemplo 16: Diseña un algoritmo que lea una calificación entera de 0 a 10 y muestre ese valor en forma de texto: "one", "two", .... "ten".

e.g. 16: Design an algorithm that reads an integer grade from 0 to 10 and display that value in textual form: "one", "two", .... "ten".

**Flowchart**



**Pseudocode**

```
int grade
write "Enter a number from 1 to 10:"
read grade
case grade
  1: write "one"
  2: write "two"
  3: write "three"
  4: write "four"
  5: write "five"
  6: write "six"
  7: write "seven"
  8: write "eight"
  9: write "nine"
  10: write "ten"
  default: write "error"
endcase
```

```
import java.util.Scanner;
class Example16 {
    public static void main(String[] argv) {
        int grade;
        Scanner inputValue;
        System.out.println("Enter a number from 1 to 10:");

        inputValue = new Scanner(System.in);
        grade = inputValue.nextInt();

        switch(grade) {
            case 1: System.out.println("one");
                    break;
            case 2: System.out.println("two");
                    break;
            case 3: System.out.println("three");
                    break;
            case 4: System.out.println("four");
                    break;
            case 5: System.out.println("five");
                    break;
            case 6: System.out.println("six");
                    break;
            case 7: System.out.println("seven");
                    break;
            case 8: System.out.println("eight");
                    break;
            case 9: System.out.println("nine");
                    break;
            case 10: System.out.println("ten");
                    break;
            default: System.out.println("ERROR");
        }
    }
}
```

17.- Diseña un algoritmo que reciba horas, minutos y segundos y muestre las horas, minutos y segundos resultantes de añadir un segundo.

18.- Diseña un algoritmo que calcule el salario neto de un trabajador en función del número de horas de trabajo y los impuestos de acuerdo con las siguientes reglas:

- Las primeras 35 horas se pagan al precio normal por hora
- Las horas que superen esas 35 horas se pagan a 1,5 veces el precio normal.
- Los tipos impositivos son:
  - Los primeros 500 € están libres de impuestos.
  - Los próximos 400 € tienen una tributación del 25%
  - Y el resto, un tipo impositivo del 45%.

Los datos de entrada son:

- \* € precio por hora
- \* número de horas.

Datos de salida:

- \* Salario bruto
- \* Salario neto
- \* Impuestos

-----En inglés-----

18.- Design an algorithm that calculates the net pay for a worker depending on the number of working hours and the taxes according to the following rules:

- First 35 hours are paid at the normal price per hour
- The hours that exceed those 35 hours are paid at 1.5 times the normal price.
- Tax rates are:
  - First 500 € are tax free.
  - next 400 € have a taxation of 25%
  - And the rest a 45% tax rate.

Input data is:

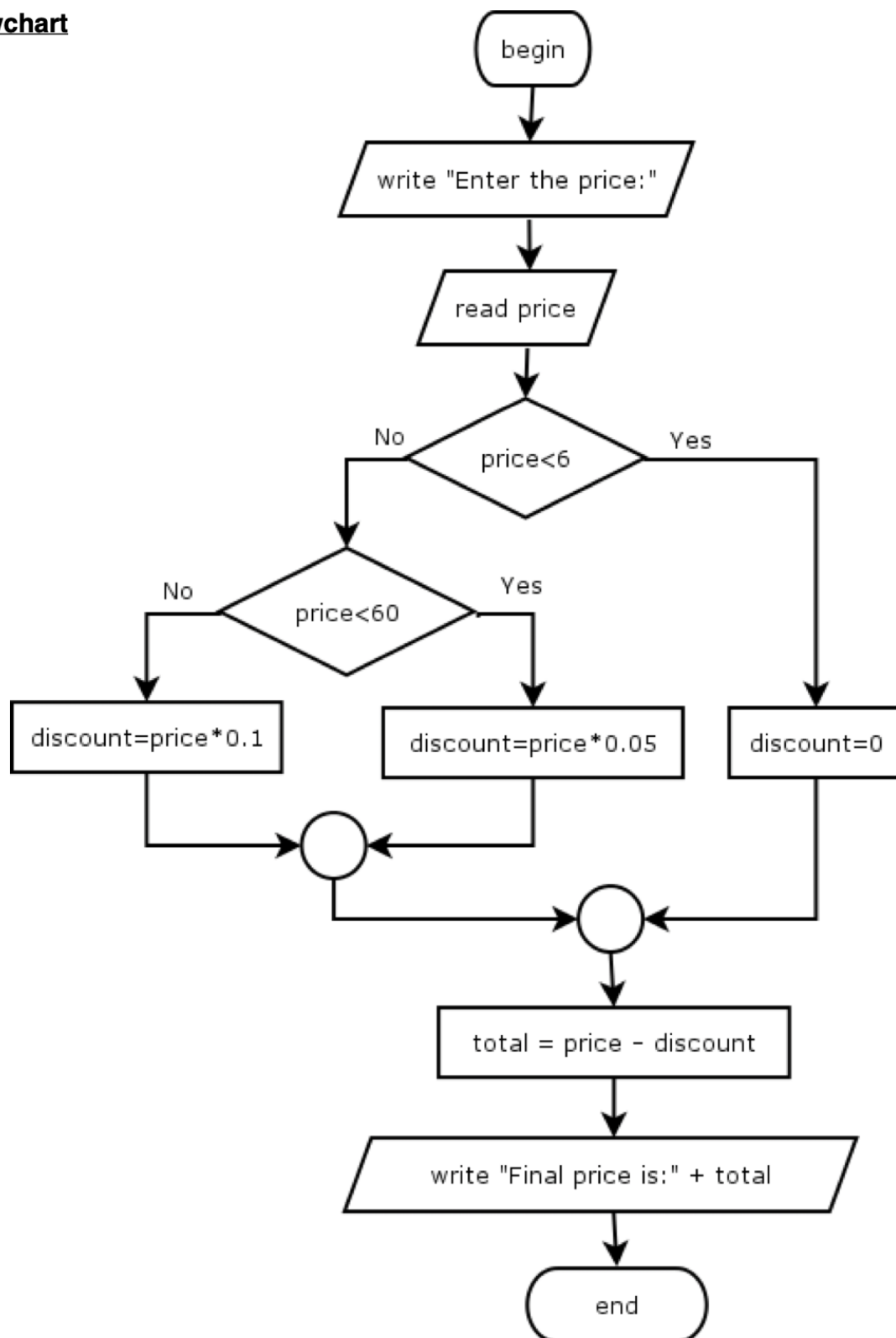
- € price per hour
- number of hours.

Output data:

- Gross pay
- Net pay
- Taxes

•

Ejemplo 19: Un determinado comercio hace un descuento en función del precio de cada producto. Si el precio es inferior a 6 euros no hay descuento. Si es mayor o igual a 6 euros e inferior a 60 €, entonces se aplica un 5% de descuento, y si es mayor o igual a 60 € aplicamos un 10% de descuento. Diseña el algoritmo para calcular el precio final.

**Flowchart**

**Pseudocode**

```
float price, discount, total
write "Enter the price:"
read price
if price < 6
    discount = 0
else
    if price < 60
        discount = price * 0.05
    else
        discount = price * 0.1
    endif
endif
total = price - discount
write "Final price is " total
```



```
import java.util.Scanner;
class Example19 {
    public static void main(String[] argv) {
        float price, discount, total;
        System.out.println("Enter the price:");

        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        price = inputValue.nextFloat();

        if (price < 6) {
            discount = 0;
        } else {
            if (price < 60) {
                discount = price * 0.05f;
            } else {
                discount = price * 0.1f;
            }
        }
        total = price - discount;
        System.out.println("Final price is "+total);
    }
}
```

Ejercicio para que hagas:

20.- Diseña un algoritmo que lea un año como datos de entrada y muestre si es un año bisiesto. Todos los múltiplos de 4, excepto los que son múltiplos de 100 y no de 400 son años bisiestos. (Ej. Años bisiestos: 1600, 2000, 2400. No bisiestos: 1700, 1800, 1900...)

## 2.4. Estructuras iterativas (bucles).



**Las estructuras iterativas** alteran el flujo normal de ejecución de un algoritmo, haciendo posible que un bloque de acciones o instrucciones se ejecute un cierto número de veces, dependiendo del cumplimiento de una condición. Veremos 3 tipos de iteraciones:

- **While** (mientras)
- **Do-While** (haz mientras)
- **For** (para)

### 2.4.1. Estructura WHILE

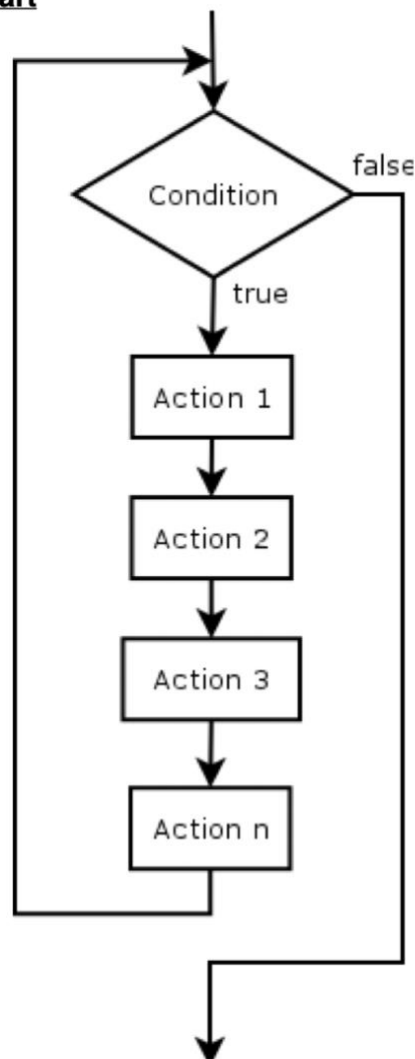
Estructura **while**: se ejecuta un bloque de acciones mientras una condición se evalúe como verdadera. La condición siempre se evalúa antes de entrar en el bucle, lo que hace posible que el bloque de acciones nunca se ejecute si la condición se evalúa como falsa al principio.

No sabemos de antemano el número de veces que se ejecutará el bucle.

#### Pseudocode

```
while condition do
  instruction 1
  instruction 2
  ...
  instruction n
endwhile
```

#### Flowchart



## 2.4.2. Estructura DO-WHILE.

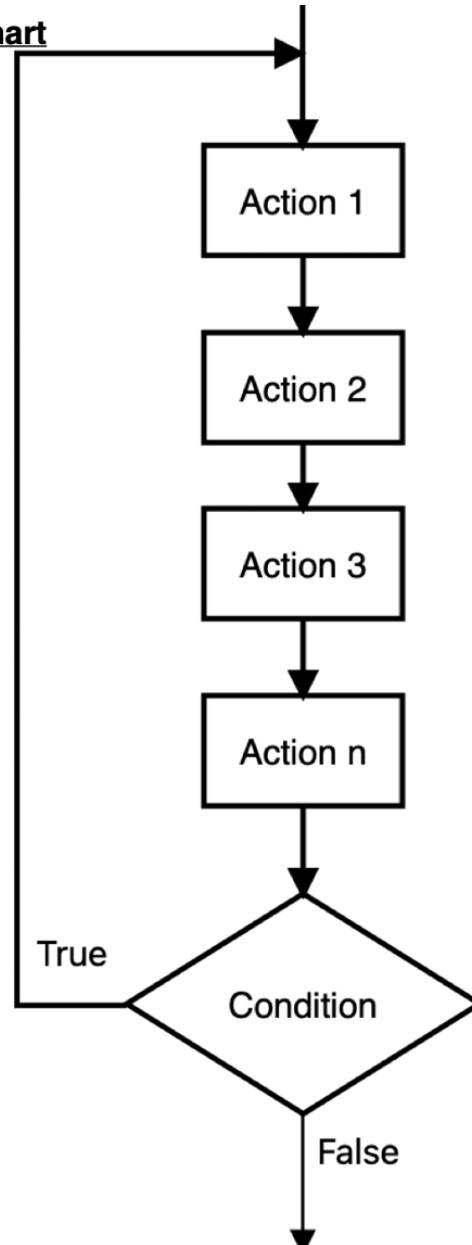
Estructura **Do-while**: se ejecuta un bloque de acciones mientras una condición se evalúa como verdadera. La condición siempre se evalúa al final del bucle, lo que hace que el bloque de acciones se ejecute al menos una vez.

No sabemos de antemano el número de veces que se ejecutará el bucle.

### Pseudocode

```
do
  instruction 1
  instruction 2
  ...
  instruction n
While condition
```

### Flowchart

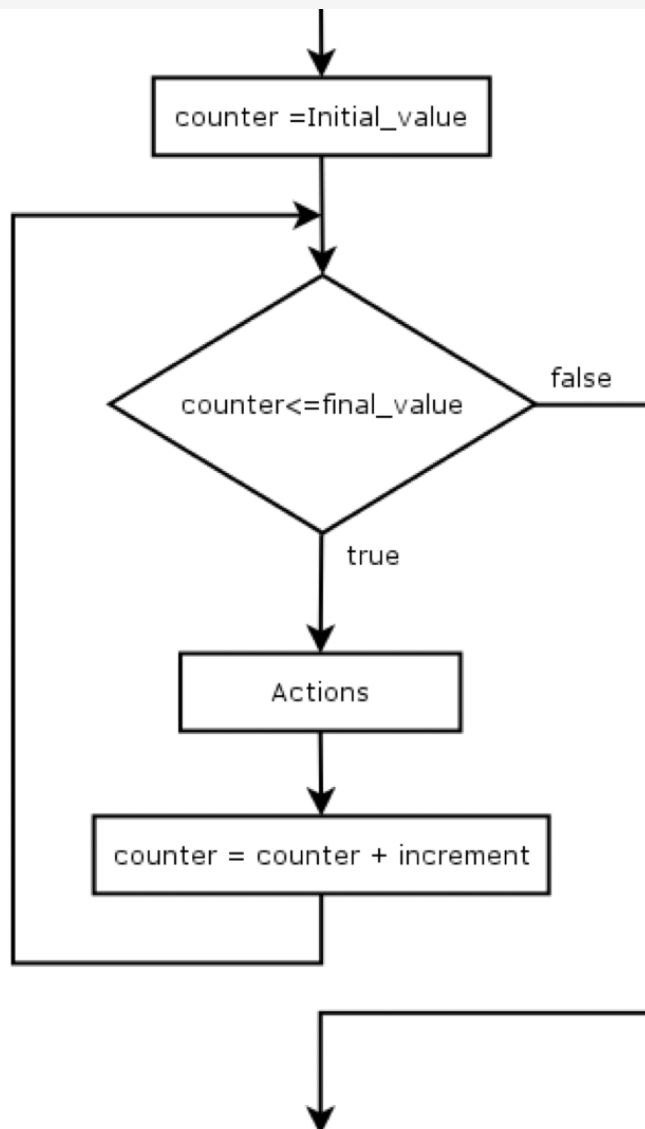


### 2.4.3. Estructura FOR.

Estructura **For**: En este tipo de bucle se conoce de antemano el número de veces que se ejecutará el bloque de instrucciones. Es un caso particular de una estructura While.

- A una variable llamada **counter** se le asigna un valor inicial.
- La condición se evalúa utilizando ese contador como: **counter <= final\_value**
- En cada iteración, la variable de contador se incrementa en un valor fijo (**incremento**).

#### Flowchart



#### Pseudocode

```
for counter from initial_value to final_value step by increment  
  actions  
endfor
```

#### 2.4.4. Finalización de un bucle

Uno de los peligros a la hora de implementar un bucle es que nunca termine. A esto se le llama **bucle infinito**, lo que haría que el algoritmo o programa se esté ejecutando para siempre. Para no cometer este error debemos recordar que las condiciones de los bucles deben cambiarse dentro del bloque del bucle. Por lo tanto, si usamos una variable para evaluar su valor en la condición del bucle, tenemos que cambiar su valor dentro del bloque ejecutado en cada iteración.

Hay diferentes enfoques para finalizar un bucle:

1.- Cuando sabemos el número exacto de veces que se tiene que ejecutar el bucle, utilizamos un contador:

```
counter = 1
while counter <= 10
    ...
    counter = counter + 1
endwhile
```

Nota: Podríamos haber usado un bucle FOR.

2.- Preguntando si queremos seguir en el bucle.

```
while ((again == "s") or (again == "S")) do
    ...
    write "Go on entering data?"
    read again
endwhile
```

3.- Usando un centinela

```
do
    ...
    read grade
while grade != -1
```

4.- Usando un interruptor

```
while switch == true
    ...
    En algún lugar del bucle asignaremos false a switch
endwhile
```

## 2.5. Variables de control

Son variables utilizadas por un programa que realizan una función específica y que reciben un nombre debido a su uso habitual. Ya hemos visto algunos de ellos.

### 2.5.1. Contadores.

Se utilizan para contar el número de veces que ocurre un evento. Por lo general, incrementan su valor en 1 y la mayoría de las veces comienzan en 0 o 1, aunque puede haber diferentes incrementos o decrementos.

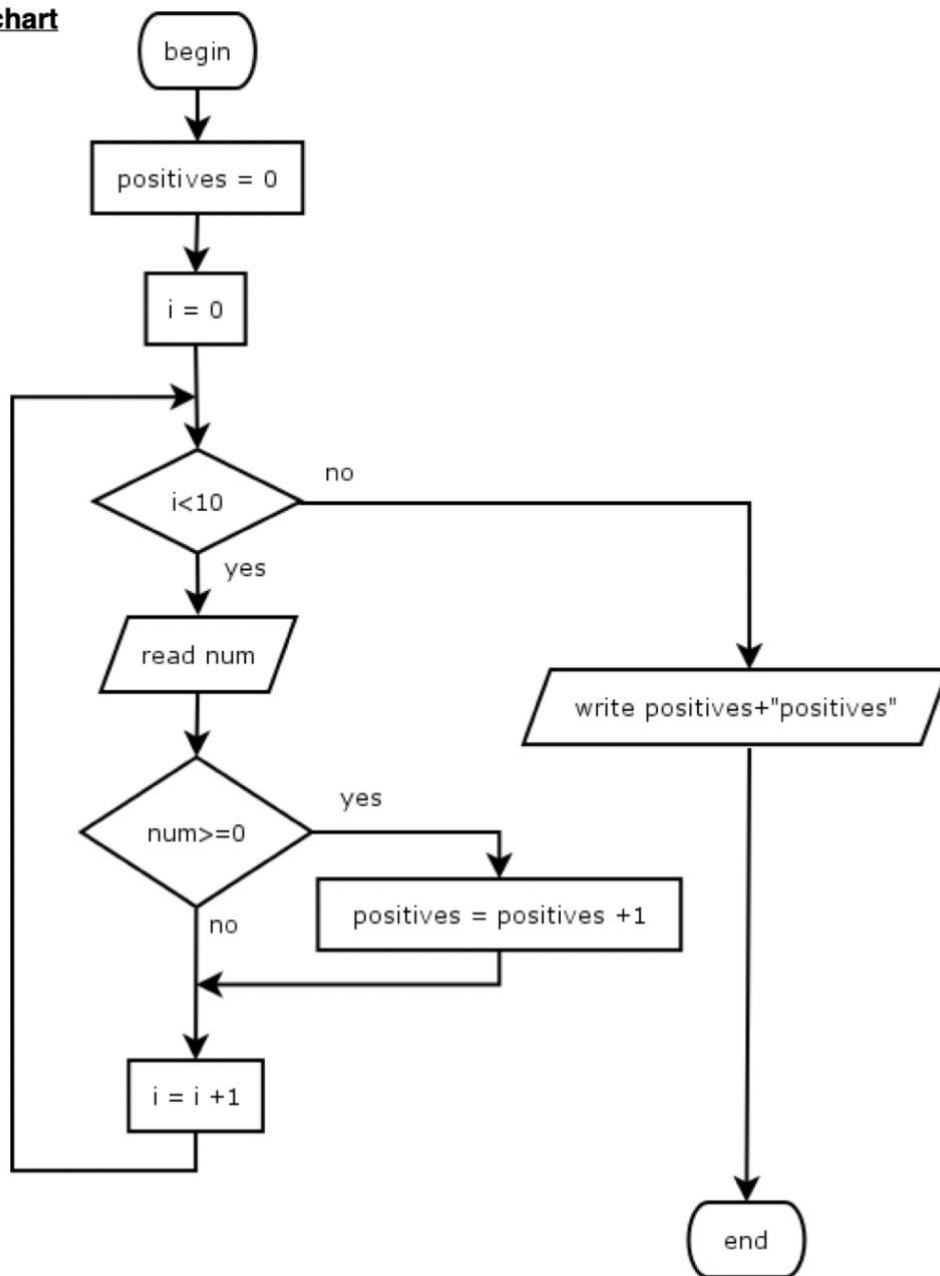
Realizamos 2 tipos de operaciones en un contador:

- **Inicialización:** lo inicializamos a 0 si realizamos un conteo natural, o a un valor inicial si queremos realizar otro tipo de conteo. O valor final si queremos hacer un conteo inverso.
- **Incremento:** Cada vez que realizamos el evento que queremos contar tenemos que incrementar el valor del contador en 1 si realizamos un conteo natural o `increment_value` si realizamos otro tipo de conteo. O -1 si estamos haciendo un conteo inverso.

Ejemplo 21: Diseña un algoritmo que lea 10 números y cuente cuántos de ellos son positivos ( $\geq 0$ )

e.g. 21: Design an algorithm that reads 10 numbers and counts how many of them are positive ( $\geq 0$ )

**Flowchart**



### **Pseudocode**

```

int positives, i, num

positives = 0
i = 0
while i < 10 do
    read num
    if num >= 0
        positives = positives + 1
    endif
    i = i + 1
endwhile
write "positives: " + positives
    
```

```

import java.util.Scanner;
class Example21 {
    public static void main(String[] argv) {
        int positives, i, num;

        //Reading the value
        Scanner inputValue;
        inputValue=new Scanner(System.in);

        i = 0;
        positives = 0;
        while (i < 10) {
            num = inputValue.nextInt();
            if (num >= 0) {
                positives = positives + 1;
            }
            i = i + 1;
        }
        System.out.println(positives + " positives");
    }
}
    
```



Usando una estructura For equivalente.

```
import java.util.Scanner;
class Example21 {
    public static void main(String[] argv) {
        int positives, num;

        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);

        positives = 0;
        for (int i = 0; i < 10; i++) {
            num = inputValue.nextInt();
            if (num >= 0) {
                positives = positives + 1;
            }
        }
        System.out.println(positives + " positives");
    }
}
```

### 2.5.2. Acumuladores

Los acumuladores se utilizan en un programa para acumular elementos sucesivos con la misma operación. Suelen utilizarse para calcular sumas y productos, aunque existen otros tipos de acumulaciones menos comunes con diferentes operaciones.

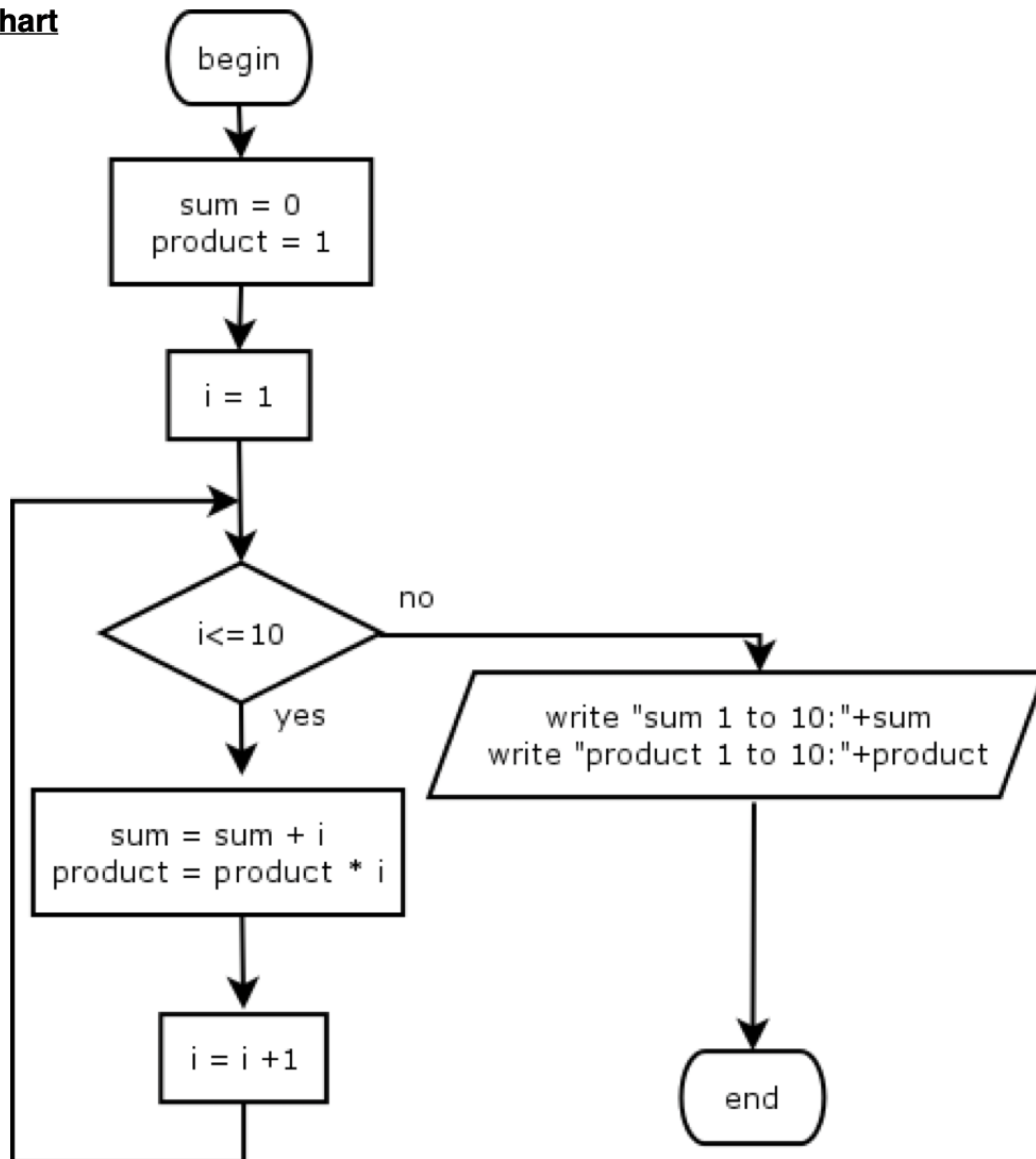
Al igual que ocurre con los contadores, para utilizarlos tenemos que realizar dos operaciones básicas:

- **Inicialización:** Normalmente inicializamos un acumulador con el elemento neutro de la operación utilizada para acumular. 0 para la suma y 1 para el producto.
- **Acumulación:** Cada vez que conseguimos un elemento a acumular realizamos la operación utilizando el nuevo valor a acumular y el valor acumulado hasta el momento.

Ejemplo 22: Diseñar un algoritmo que escriba la suma de los 10 primeros números naturales y el producto de los 10 primeros números naturales.

e.g. 22: Design an algorithm that writes the sum of the first 10 natural numbers and the product of the 10 first natural numbers

### Flowchart



### Pseudocode

```

int sum, product

sum = 0
product = 1
for i from 1 to 10 do
    sum = sum + i
    product = product * i
endfor
write "sum from 1 to 10:" + sum
write "product from 1 to 10:" + product
    
```

```
class Example22 {  
    public static void main(String[] argv) {  
        int sum, product;  
        sum = 0;  
        product = 1;  
        for (int i = 1; i <= 10; i++) {  
            sum = sum + i;  
            product = product * i;  
        }  
        System.out.println("Sum from 1 to 10:" + sum);  
        System.out.println("Product from 1 to 10:" + product);  
    }  
}
```

Ejercicio para que hagas:

Escribe el pseudocódigo utilizando la estructura While equivalente.

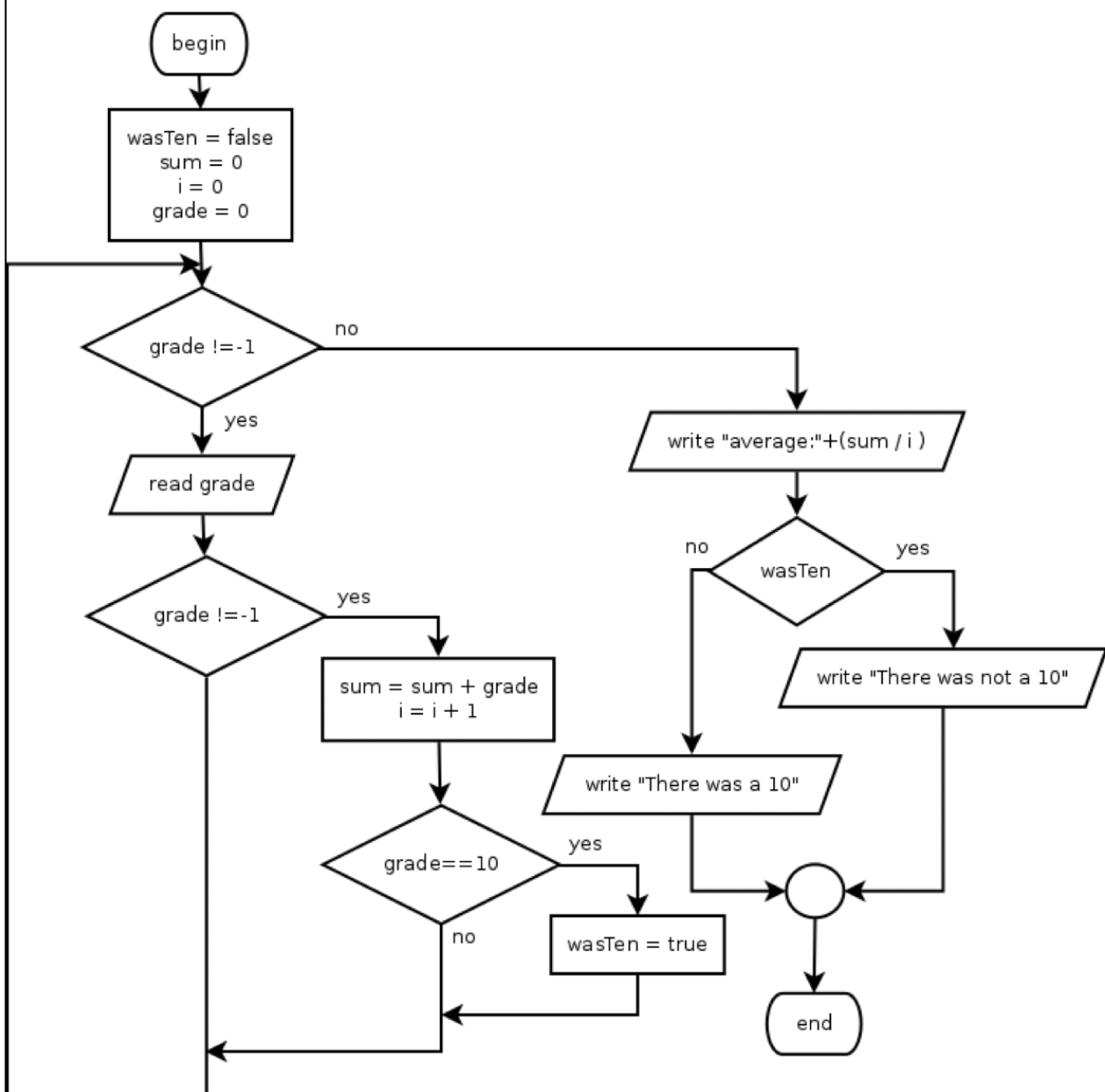
### 2.5.3. Interruptores.

Los interruptores o switches son variables booleanas. Solo pueden contener los valores verdadero o falso (Sí/No, 1/0). Llevan la información de un lugar a otro en el programa. Actúan como recordatorios de algo que sucedió durante la ejecución del programa.

Ejemplo 23: Diseña un algoritmo que lea un conjunto de calificaciones desde el teclado hasta que se ingrese un -1 y escribe la calificación promedio y si hubo un 10 o no.

e.g. 23: Design an algorithm that reads a set of grades form the keyboard until a -1 is entered and it writes the average grade and if there was a 10 or not.

#### Flowchart



### **Pseudocode**

```
float grade, sum
boolean wasTen
int i

sum = 0
i = 0
grade = 0
wasTen = false
while grade != -1 do
    read grade
    if grade != -1
        sum = sum + grade
        i = i + 1
        if grade == 10
            wasTen = true
        endif
    endif
endwhile
write "average:" + sum / i
if wasTen
    write "There was a ten"
else
    write "There was not a ten"
endif
```

```
import java.util.Scanner;
class Example23 {
    public static void main(String[] argv) {
        float grade = 0, sum = 0;
        boolean wasTen = false;
        int i = 0;

        Scanner inputValue;
        inputValue = new Scanner(System.in);

        while (grade != -1) {
            grade = inputValue.nextFloat();
            if (grade != -1) {
                sum = sum + grade;
                i = i + 1;
                if (grade == 10) {
                    wasTen = true;
                }
            }
        }
        System.out.println("Average: " + sum / i);
        if (wasTen) {
            System.out.println("There was a 10");
        } else {
            System.out.println("There was not a 10");
        }
    }
}
```

Ejercicios:

- 24.- Diseña un algoritmo para calcular el factorial de un número
- 25.- Diseña un algoritmo que lea un número natural y escriba toda su tabla de multiplicar.
- 26.- Diseña un algoritmo que lea un número natural y escriba sus divisores.
- 27.- Diseña un algoritmo que escriba los primeros 40 términos de la serie de Fibonacci.
- 28.- Diseña un algoritmo que calcule un producto de 2 números > 0 utilizando sumas sucesivas.
- 29.- Algoritmo que calcula el resto de la división de enteros mediante sucesivas restas.
- 30.- Diseña un algoritmo que lea un número n y luego imprima esto:

```
1
1 2
1 2 3
...
1 2 3 4 5 ... n
```

31.- Diseña un algoritmo que lea un número n y luego imprima este (Ejemplo con n = 5):

```

/ / / / 1 / / / /
/ / / 2 1 2 / / /
/ / 3 2 1 2 3 / /
/ 4 3 2 1 2 3 4 /
5 4 3 2 1 2 3 4 5
    
```

32.- Calcula el valor de la raíz cuadrada para un número n usando la solución iterativa:

Dado x como valor estimado (puede establecer x = n al principio), cada iteración se calcula como:

$$y = \frac{1}{2} \left( x + \frac{n}{x} \right)$$

Luego haz x = y y realiza otra iteración.

Calcula la raíz cuadrada de n para 10 iteraciones.

(<http://www.slideshare.net/rjvillon/2metodo-iterativo>)

33.- Ahora modifica el ejercicio anterior y deja de iterar cuando  $|x - y| < 0,000001$ . No se puede utilizar la función Math.abs( ).

34.- Calcula el valor e utilizando la serie de Newton. Los datos de entrada son un número n. Usa un valor como 10 para el infinito (prueba con varios números):

$$e = \sum_{n=0}^{\infty} \left( \frac{1}{n!} \right)$$

<http://www.intmath.com/exponential-logarithmic-functions/calculating-e.php>

Pista para el ejercicio. 34:  $e = 1/1 + 1/1 + 1/2 + 1/6 + 1/24 + 1/120 + \dots$